SIMATS ENGINEERING

CSA0823 - PYTHON PROGRAMMING

ASSIGNMENT 4 (CO4)

ROBUST DATA PROCESSING WITH FILES, EXCEPTIONS, MODULES AND PACKAGES

Course Outcome: CO4 – Design and implement Python programs using file handling, modules and packages to process and analyse data effectively.

Submitted by: D.S. Tharuniya Priya
Register Number: 192511317
Programme: B.E. Computer Science and Engineering
Year: II Year
Department: Computer Science and Engineering
Institution: SIMATS Engineering

Submitted to: Hariramachandran

GitHub Repository Link: https://github.com/tharu-ui/CSA0823---Python-Programing/tree/main/C04%20Assignment%204

TASK 1 – FILE-PROCESSING DESIGN REVIEW

Bloom's Level: K4 – Analyse
Marks: 6

Task 1.1 – Diagnose the Script

The given program is intended to read student mess-bill records from a text file, calculate the total amount and display the bill details. However, the program contains several design deficiencies that may cause incorrect results, data loss or difficulty in maintaining the application.

Table 1.1 – Deficiencies in the Given Program

S.No.	Design deficiency	Explanation	Possible consequence in the hostel office
1	Hard-coded file path	The file path is directly written inside the program.	The program may fail when executed on another computer or when the file is stored in a different location.
2	File is not properly closed	The file is opened using open() but is not closed explicitly.	System resources may remain occupied, especially when the program processes many files.
3	Entire file is read at once	The read() method loads the complete file into memory.	Memory consumption increases for large files containing thousands of student records.
4	Bare except statement	The program catches every exception without identifying its type.	Important errors such as permission problems or programming mistakes are silently ignored.
5	Use of pass inside exception handling	The program ignores invalid records without recording them.	The office cannot identify which student's bill was rejected or why it was rejected.

6	Incorrect numeric conversion	The program uses int() to convert the amount.	Amounts containing paise, such as 1250.50, cannot be processed correctly.
7	Repeated string concatenation	The report is constructed using repeated + operations.	Repeated string creation is inefficient for large reports and makes the code less maintainable.
8	Empty lines are not handled explicitly	Splitting the complete file using newline characters may produce an empty final record.	The program may attempt to process an empty record and silently ignore the resulting error.
9	No rejected-record report	Invalid records are not stored separately.	The office cannot verify missing or incorrect bill entries.
10	No reusable function	The file name is fixed within the program.	The same program cannot conveniently process different monthly bill files.

Explanation

The given script is unsuitable for reliable office use because it does not distinguish between valid and invalid records. It may silently ignore incorrect data and produce an incomplete total. It also assumes that the file will always exist at the same location and that every amount is an integer.

A reliable file-processing program must validate records, handle specific exceptions, close files safely and preserve information about rejected data.

Task 1.2 – Repair the Design

The corrected program uses a function that accepts the file name as a parameter. It reads the file line by line using a with statement, validates each record and stores rejected records in a separate list.

Design decisions

1. The with statement automatically closes the file after processing.
2. The file is processed one line at a time to reduce memory usage.
3. The Decimal data type is used to accurately represent monetary values.
4. ValueError is handled specifically for invalid amounts.
5. InvalidOperation is handled for invalid decimal values.
6. FileNotFoundError is handled when the input file is unavailable.
7. Rejected records are stored with their line numbers and reasons.
8. The file name is passed as a function argument to improve reusability.

Corrected Python Program

```
from decimal import Decimal, InvalidOperation

def process_mess_bill(filename):
    total = Decimal("0.00")
    report = []
    rejected = []

    try:
        with open(filename, "r", encoding="utf-8") as file:
            for line_number, line in enumerate(file, start=1):
                line = line.strip()

                if not line:
                    continue

                parts = line.split(",")

                if len(parts) != 3:
                    rejected.append(
                        (line_number, line, "expected 3 fields")
```

```
            )
            continue

        register_number, name, amount = parts

        try:
            bill_amount = Decimal(amount)

            if not bill_amount.is_finite():
                raise InvalidOperation

            if bill_amount < 0:
                raise ValueError("negative amount")

        except (InvalidOperation, ValueError):
            rejected.append(
                (line_number, line, "invalid amount")
            )
            continue

        total += bill_amount
        report.append(f"{name} {bill_amount:.2f}")

except FileNotFoundError:
    print("Error: input file not found.")
    return

print("Mess Bill Report")

for record in report:
    print(record)
```

```
    print(f"Total: {total:.2f}")

    print("Rejected Lines:")

    for line_number, record, reason in rejected:
        print(f"Line {line_number}: {record} ({reason})")


if __name__ == "__main__":
    process_mess_bill("mess_oct.txt")
```

Sample Input File – mess_oct.txt

192311001,Anu,1250.50

192311002,Bala,980.00

192311003,Charu,1100.75

192311004,Deepa,invalid

192311005,Esha,750.00

wrong,line

Actual Execution Output

Mess Bill Report

Anu 1250.50

Bala 980.00

Charu 1100.75

Esha 750.00

Total: 4081.25

Rejected Lines:

Line 4: 192311004,Deepa,invalid (invalid amount)

Line 6: wrong,line (expected 3 fields)

Table 1.2 – Record Processing Results

S.No.	Register Number	Name	Amount	Processing status
1	192311001	Anu	1250.50	Accepted
2	192311002	Bala	980.00	Accepted
3	192311003	Charu	1100.75	Accepted
4	192311004	Deepa	invalid	Rejected – invalid amount
5	192311005	Esha	750.00	Accepted
6	—	—	—	Rejected – incorrect number of fields

Table 1.3 – Summary of Mess-Bill Processing

Description	Result
Total input records	6
Valid records	4
Rejected records	2
Total bill amount	₹4,081.25

Calculation

Total amount:

1250.50 + 980.00 + 1100.75 + 750.00

Total = ₹4,081.25

Task 1.3 – Design Defence

The corrected program is safer for processing a file containing 50,000 lines because it reads one record at a time instead of loading the entire file into memory. The with statement ensures that the file is closed automatically. Specific exceptions help identify errors accurately, while rejected records are preserved for later verification. The use of Decimal supports accurate monetary calculations. Passing the file name as a parameter also makes the program reusable for different monthly files.

Alternative Concept and Complexity Analysis

An alternative approach is to read the entire file using read() or readlines() before processing it.

Alternative implementation:

```
with open(filename, "r", encoding="utf-8") as file:
    lines = file.readlines()


for line in lines:
    # Validate and process each record
    Pass
```

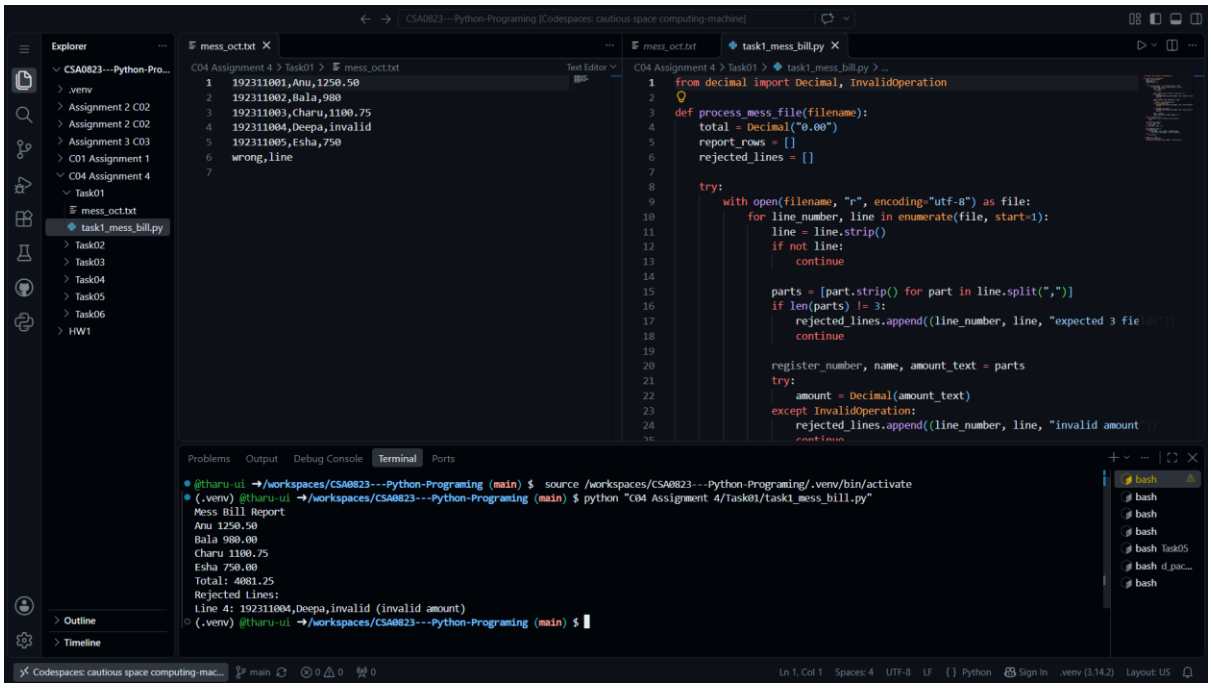
Table 1.4 – Comparison of File-Reading Approaches

Comparison	**Line-by-line reading**	**read() / readlines()**
Processing method	Reads one line at a time	Reads the complete file
Time complexity	O(n)	O(n)
Extra memory	O(L)	O(N)
Memory requirement	Stores approximately one line at a time	Stores the entire file or all lines
Large-file suitability	More suitable	Requires more memory
Processing style	Sequential	Entire file loaded before processing

Here, N represents the total input size and L represents the maximum size of one line.

For a file containing 50,000 lines, both approaches require O(n) time to examine all records. However, line-by-line processing requires significantly less additional memory because it does not store the entire file.

Conclusion: Line-by-line processing is more appropriate for large files because it provides efficient memory usage and reliable resource management.



TASK 2 – FROM REQUIREMENT TO FILE-PROCESSING DESIGN

Bloom's Level: K3 – Apply
Marks: 8

Task 2.1 – Model the Data

The biometric attendance system generates one text file every day. Each record contains a student's register number, attendance date and attendance status.

The attendance status is represented by:

- P – Present
- A – Absent

The system must process all daily files for a month and generate an attendance percentage report, a list of students below 75% attendance and a damaged-record report.

Input File Format

Each line follows the format:

RegisterNumber,Date,Status

Example:

192311001,2026-10-08,P

192311002,2026-10-08,A

192311003,2026-10-08,P

Data Structures Used

A dictionary is used to store attendance information for each student.

Example:

```
attendance = {
   "192311001": {"present": 0, "absent": 0},
   "192311002": {"present": 0, "absent": 0}
}
```

The dictionary uses the register number as the key. Each student's present and absent counts are stored as values.

A separate list is used to store damaged records.

Example:

```
damaged_records = [
   ("day02.txt", 3, "damaged attendance line")
]
```

Table 2.1 – Input and Output Data Model

Component	Description	Example
Register number	Unique student identifier	192311001
Date	Date of attendance	2026-10-08
Status	Attendance status	P or A
Dictionary	Stores student-wise attendance counts	Present = 2, Absent = 1
Damaged-record list	Stores rejected records	File name, line number and reason
Attendance report	Stores monthly attendance percentage	attendance_percentage.csv
Below-75 report	Lists students below the required percentage	below_75.txt
Error report	Stores damaged records	damaged_lines.csv

Attendance Percentage Formula

$$\text{Attendance Percentage} = \frac{\text{Present Days}}{\text{Present Days} + \text{Absent Days}} \times 100$$

Where:

$$\text{Total Working Days} = P + A$$

Task 2.2 – Design the File Operations

The system uses different file modes depending on the purpose of each operation.

Table 2.2 – File Modes and Their Uses

File mode	Purpose	Application in the attendance system
r	Read an existing file	Reads daily biometric attendance files
w	Write a new file or overwrite an existing file	Creates the monthly attendance report
a	Append data to an existing file	Adds new error details without deleting previous entries

Table 2.3 – File Methods and Their Uses

Method	Purpose	Reason for use
open()	Opens a file	Provides access to input and output files
read()	Reads the entire file	Useful for small files when complete content is required
readline()	Reads one line	Useful when individual lines must be processed
File iteration	Reads lines sequentially	Efficient for processing large attendance files
write()	Writes content into a file	Used to generate reports and error logs
close()	Closes an opened file	Releases the file resource
strip()	Removes unnecessary surrounding whitespace	Cleans input records
split()	Separates fields using a delimiter	Extracts register number, date and status

Encoding

The encoding used is **UTF-8**.

open(filename, "r", encoding="utf-8")

UTF-8 supports standard text characters and provides consistent file handling across different systems.

The program uses the pathlib or os module to identify all daily attendance files stored in a folder.

Task 2.3 – Design the Processing

Processing Algorithm

1. Identify the folder containing the daily attendance files.
2. List all files in the folder.
3. Initialize an empty attendance dictionary.
4. Initialize an empty list for damaged records.
5. Open each daily file using read mode.
6. Read the file line by line.
7. Split each line into three fields.
8. Validate the number of fields and attendance status.
9. If valid, update the student's attendance count.
10. If invalid, store the file name, line number and reason.
11. Calculate the attendance percentage for each student.
12. Write the attendance percentage report.
13. Identify students whose attendance is below 75%.
14. Write the below-75% report.
15. Write all damaged records into a separate CSV file.

16. Display the processing summary.

Algorithm: PROCESS_MONTHLY_ATTENDANCE

Input: Daily attendance files stored in a folder.

Output: Attendance percentage report, below-75% report and damaged-record report.

BEGIN

INITIALIZE attendance_dictionary
INITIALIZE damaged_records_list

GET all daily files from the attendance folder

FOR each file in daily_files DO

OPEN file in READ mode

SET line_number = 0

FOR each line in file DO

INCREMENT line_number

REMOVE unnecessary spaces from line
SPLIT line using comma

IF number of fields is not equal to 3 THEN
ADD file name, line number and error reason
TO damaged_records_list

ELSE

```
        EXTRACT register_number, date and status

        IF status is not P or A THEN
          ADD file name, line number and error reason
          TO damaged_records_list

        ELSE
          IF register_number is not in attendance_dictionary THEN
            INITIALIZE present and absent counts
          END IF

          IF status = P THEN
            INCREMENT present count
          ELSE
            INCREMENT absent count
          END IF
        END IF
      END IF

    END FOR

    CLOSE file

END FOR

OPEN attendance report in WRITE mode

FOR each student in attendance_dictionary DO

    CALCULATE total working days
    CALCULATE attendance percentage
```

```
        WRITE student details and percentage

        IF attendance percentage < 75 THEN
            ADD student to below-75% report
        END IF

    END FOR

    CLOSE attendance report

    WRITE below-75% report
    WRITE damaged-record report

    DISPLAY processing summary

END
```

Output Files

S.No.	Output file	Purpose
1	attendance_percentage.csv	Stores present days, absent days and attendance percentage
2	below_75.txt	Lists students whose attendance is below 75%
3	damaged_lines.csv	Stores damaged records with file name and line number

Task 2.4 – Trace

The program was executed using three daily attendance files.

Table 2.4 – Attendance Processing Results

Register Number	Present Days	Absent Days	Total Days	Attendance Percentage
192311001	2	1	3	66.67%
192311002	1	2	3	33.33%
192311003	2	1	3	66.67%

Table 2.5 – Damaged Record Trace

File name	Line number	Damaged data	Reason
day02.txt	3	damaged attendance line	Expected 3 fields

Actual Execution Summary

Daily files processed: 3

Students processed: 3

Damaged records: 1

Attendance Percentage CSV

Register Number,Present,Absent,Percentage

192311001,2,1,66.67

192311002,1,2,33.33

192311003,2,1,66.67

Actual Below-75% Report

Students below 75% attendance

192311001: 66.67%

192311002: 33.33%

192311003: 66.67%

Actual Damaged-Record Report

File,Line Number,Rejected Data,Reason

day02.txt,3,damaged attendance line,expected 3 fields

All three students were correctly identified as having attendance below 75%.

Alternative Concept and Complexity Analysis

An alternative method is to calculate attendance separately for every student by reopening and scanning every daily file.

Table 2.6 – Comparison of Attendance Processing Methods

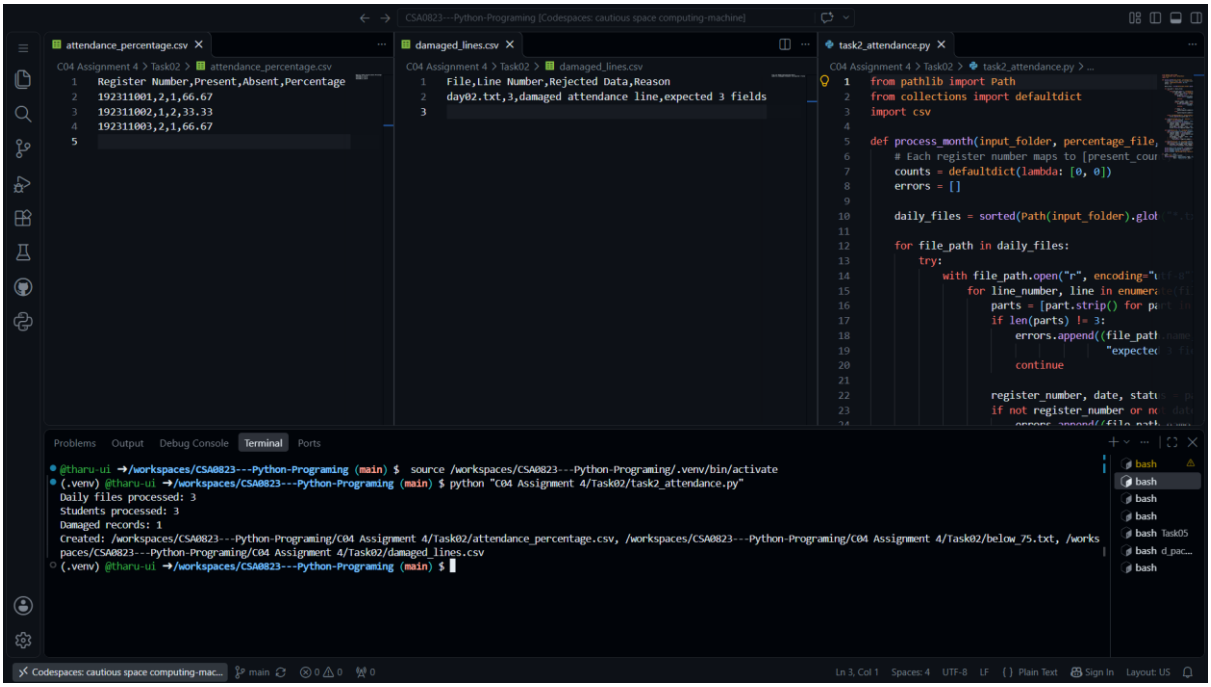
Comparison	**Dictionary-based method**	**Re-reading files for every student**
Processing method	Reads each file once	Reads all files separately for each student
Number of file scans	One complete scan	One complete scan per student
Time complexity	O(f × l)	O(s × f × l)
Extra storage	O(s) for attendance counts	O(1) additional counting space, excluding temporary data
Efficiency	More efficient	Less efficient
Suitable for large systems	Yes	Not recommended

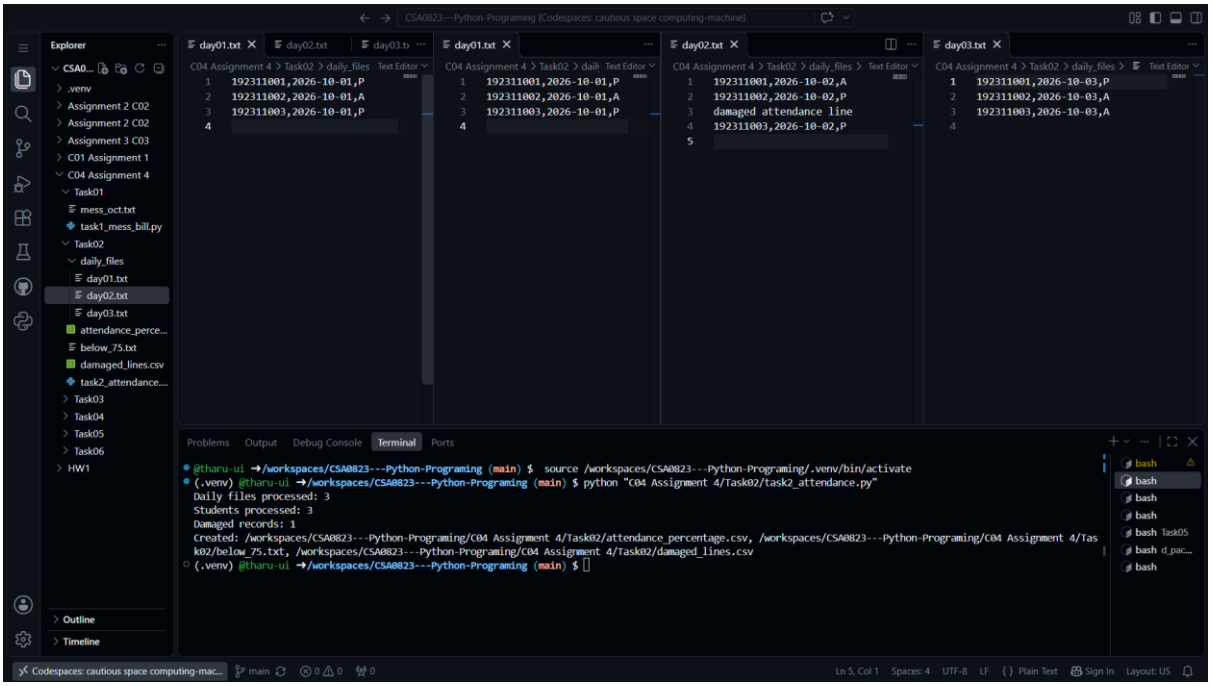
Where:

- s = number of students
- f = number of daily files
- l = average number of lines in each file

The dictionary-based method avoids repeated file reading. Therefore, it is more efficient when the number of students and daily files increases.

Conclusion: Reading each daily file once and accumulating attendance counts in a dictionary reduces repeated operations and improves processing efficiency.





TASK 3 – DESIGN THE ERROR HANDLING

Bloom's Level: K4 – Analyse
Marks: 9

Task 3.1 – Identify the Failures

File-processing applications may encounter different errors when accessing files or processing records. These errors must be identified and handled using appropriate exception classes.

Table 3.1 – Failure Analysis

S.No.	Failure condition	Possible exception	Explanation	Handling method
1	Missing folder	FileNotFoundError	The specified folder does not exist.	Verify the folder path and report the error.
2	Missing file	FileNotFoundError	The required attendance file is unavailable.	Catch the exception and display a meaningful message.
3	Permission denied	PermissionError	The program does not have permission to access the file.	Report the permission problem.
4	Wrong encoding	UnicodeDecodeError	The file contains data that cannot be decoded using the selected encoding.	Report the encoding issue or use the correct encoding.
5	Damaged record	Custom InvalidRecord exception	A record contains an incorrect number of fields.	Reject the individual record and continue processing.

6	Invalid attendance status	Custom InvalidRecord exception	The status contains a character other than P or A.	Reject the record and record its line number.
7	Invalid numeric data	ValueError	A value cannot be converted into the required numeric type.	Validate the value and report the error.
8	Invalid list indexing	IndexError	The program attempts to access a field that does not exist.	Validate the number of fields before indexing.
9	Empty file	No exception by itself	The file exists but contains no records.	Handle it as an empty dataset and display an appropriate message.
10	File path refers to a directory	IsADirectoryError	The program attempts to open a directory as a file.	Validate the path before processing.

A specific exception is preferable to a general exception because it helps identify the exact cause of failure.

Task 3.2 – Design a Custom Exception

A custom exception named InvalidRecord is defined to identify records that do not follow the required attendance format.

Definition

```
class InvalidRecord(Exception):
    """Raised when an attendance record is invalid."""
    pass
```

Conditions for Raising the Exception

The exception is raised when:

1. The record does not contain exactly three fields.
2. The attendance status is not P or A.
3. The record does not follow the expected format.

Where the Exception Is Raised

The parse_line() function validates the record. If the record is incorrect, it raises the InvalidRecord exception.

Where the Exception Is Handled

The summarise() function catches the exception for each individual line. It stores the line number and error message in a list and continues processing the remaining records.

Table 3.2 – Custom Exception Design

Function	Responsibility
InvalidRecord	Defines the custom exception
parse_line()	Validates the record and raises the exception
summarise()	Catches the exception and stores rejected-record details
Main program	Displays the result and handles file-level errors

Task 3.3 – Use the Complete try Statement

Python provides four important components for structured exception handling.

Table 3.3 – Components of Exception Handling

Component	Purpose	Application
try	Contains statements that may cause exceptions	Opens and processes attendance files

except	Handles specified exceptions	Handles missing files and invalid records
else	Executes when no exception occurs in the associated try block	Displays successful processing information
finally	Executes whether an exception occurs or not	Displays completion information and performs cleanup

Use of raise ... from ...

The statement raise NewException(...) from original_exception explicitly connects a new exception to the original exception.

This preserves the original cause and makes debugging easier.

Example:

```
try:
    with open(path, "r", encoding="utf-8") as file:
        data = file.read()


except FileNotFoundError as error:
    raise RuntimeError("Unable to process attendance file") from error
```

The resulting RuntimeError explains the application-level problem, while the original FileNotFoundError remains available as its cause.

Completed Program – TODO 1 and TODO 2

```
import os
import tempfile


class InvalidRecord(Exception):
```

```
    """Raised when a line of the attendance file is invalid."""


def parse_line(line):
    parts = line.strip().split(",")

    if len(parts) != 3:
        raise InvalidRecord(
            f"expected 3 fields, found {len(parts)}"
        )

    register_number, date, status = parts

    if status not in ("P", "A"):
        raise InvalidRecord(f"invalid status '{status}'")

    return register_number, date, status


def summarise(path):
    present, absent, errors = 0, 0, []

    try:
        with open(path, "r", encoding="utf-8") as file:
            for line_number, line in enumerate(file, start=1):

                try:
                    register_number, date, status = parse_line(line)

                    if status == "P":
                        present += 1
                    else:
```

```
                    absent += 1

            except InvalidRecord as error:
                errors.append((line_number, str(error)))

    except FileNotFoundError as error:
        raise RuntimeError(
            f"attendance file not found: {path}"
        ) from error

    except PermissionError as error:
        raise RuntimeError(
            f"permission denied: {path}"
        ) from error

    except UnicodeDecodeError as error:
        raise RuntimeError(
            f"invalid file encoding: {path}"
        ) from error

    else:
        print("File processed successfully.")

    finally:
        print("Attendance processing attempt completed.")

    return present, absent, errors


data = (
    "192311001,2026-10-08,P\n"
    "192311002,2026-10-08,A\n"
```

```
    "192311003,2026-10-08\n"
    "192311004,2026-10-08,L\n"
)

with tempfile.NamedTemporaryFile(
    "w",
    suffix=".csv",
    delete=False,
    encoding="utf-8"
) as tmp:
    tmp.write(data)

print(summarise(tmp.name))
os.remove(tmp.name)
```

Expected and Actual Output

File processed successfully.

Attendance processing attempt completed.

(1, 1, [(3, 'expected 3 fields, found 2'), (4, "invalid status 'L'")])

Table 3.4 – Execution Trace

Line number	**Input record**	**Validation result**	**Action**
1	192311001,2026-10-08,P	Valid	Present count increased to 1
2	192311002,2026-10-08,A	Valid	Absent count increased to 1
3	192311003,2026-10-08	Invalid – only 2 fields	Added to error list

4	192311004,2026-10-08,L	Invalid – incorrect status	Added to error list

Missing File Test

When the file name is changed to a file that does not exist, the program raises a FileNotFoundError. The summarise() function catches it and raises a RuntimeError using raise ... from ..., preserving the original cause.

The main program can handle the resulting exception as follows:

```
try:
    print(summarise("file_that_does_not_exist.csv"))

except RuntimeError as error:
    print("Handled:", error)
```

Actual Missing-File Output

Attendance processing attempt completed.

Handled: attendance file not found: file_that_does_not_exist.csv

Task 3.4 – Misuse Experiment

The experiment replaces specific exception handling with a general exception handler.

Incorrect implementation:

```
try:
    total = presnt + 1

except Exception:
    pass
```

The variable presnt is incorrectly typed instead of present. Python raises a NameError because the variable is not defined.

However, except Exception: pass catches the error and ignores it.

Table 3.5 – Misuse Experiment Results

S.No.	Experiment	Actual behaviour	Consequence
1	Specific exception handling	Identifies the actual error	Easier debugging
2	General except Exception	Catches many different exceptions	Error information becomes less specific
3	except Exception: pass	Silently ignores the error	Program may continue with incorrect results
4	Typing error in variable name	Produces NameError	The error is hidden when ignored

Explanation

Using except Exception: pass is dangerous because it hides programming errors and unexpected failures. The program may appear to execute successfully even though important operations were not completed. Specific exception handling is more reliable because it makes errors visible and allows suitable corrective actions.

Alternative Concept and Complexity Analysis

An alternative approach is to validate every record using conditional statements before processing it.

Example:

```
parts = line.strip().split(",")


if len(parts) == 3:
    if parts[2] in ("P", "A"):
        # Process valid record
```

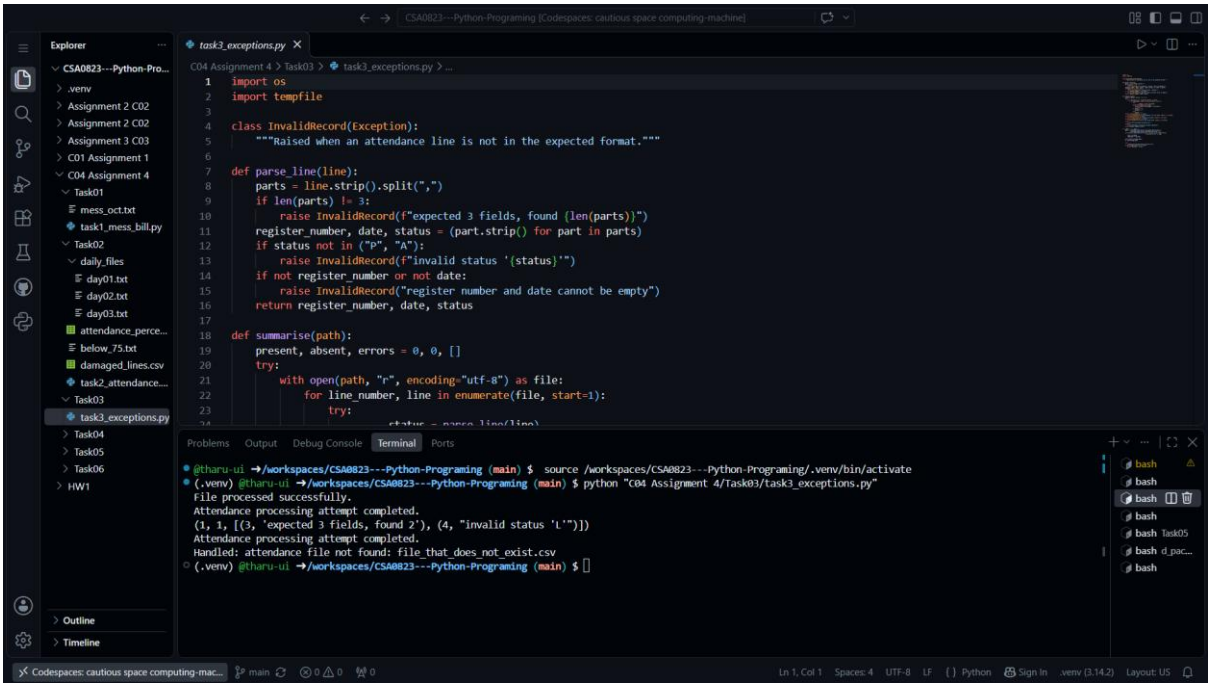
```
        pass
    else:
        # Reject invalid status
        pass
else:
    # Reject incorrect field count
    Pass
```

Table 3.6 – Comparison of Exception-Based and Conditional Validation

Comparison	**Exception-based approach**	**Conditional validation**
Validation method	Raises and catches exceptions	Uses if statements
Normal valid record	Processes without an exception	Checks conditions before processing
Invalid record	Raises and handles an exception	Executes the rejection branch
When errors are rare	Efficient and readable	Performs validation checks for every record
When errors are common	Exception handling overhead increases	Conditional checks are generally more suitable
Time per valid line	Approximately O(1)	O(1)
Time per invalid line	Higher due to exception creation and handling	O(1) conditional checks
Overall time	O(n)	O(n)
Best application	Unexpected or exceptional failures	Expected input validation

Both methods have O(n) overall time complexity for n records. However, exception-based validation is generally preferable when invalid records are rare. Conditional validation is usually more suitable when invalid input is expected frequently.

Conclusion: Specific exception handling, custom exceptions and meaningful error messages improve the reliability and maintainability of file-processing programs.



TASK 4 – FILE AND EXCEPTION FORENSICS

Bloom's Level: K4 – Analyse
Marks: 7

Task 4.1 – Locate the Defects

The given library program is intended to store returned books from multiple days in a log file. However, it contains defects that cause previous records to be lost and file resources to be handled incorrectly.

Table 4.1 – Defects in the Given Program

S.No.	Defect	Explanation	Result
1	Use of write mode (w)	Write mode truncates the existing file when it is opened.	Records from previous days are deleted.
2	Missing newline characters	Each book name is written without a newline separator.	All book names may appear joined together.
3	No context manager	Files are opened without a with statement.	Files may remain open if an exception occurs before close().
4	No error log	The program only prints missing.	The error is not permanently recorded for future examination.
5	No proper resource management	The input file is opened using open() without a context manager.	The file may not be closed correctly after an unexpected failure.

Why Earlier Days Are Lost

The statement:

open("returns.log", "w")

opens the file in write mode. This removes the existing content before writing the new data. Therefore, only the most recently processed day's records remain in the log.

Why Books Are Joined Together

The statement:

log.write(b)

does not automatically insert a newline character. Therefore, multiple book names may appear on the same line.

What Happens When write() Fails?

If write() raises an exception before close() is executed, the program may leave the file open. Buffered data may also fail to be flushed correctly. A context manager or a finally block is required to ensure proper closure.

Task 4.2 – Construct Evidence

A test is performed using three daily files.

Sample Input – day1.txt

The Alchemist

Atomic Habits

Sample Input – day2.txt

White Nights

Metamorphosis

Sample Input – day3.txt

Ikigai

Wings of Fire

Testing Procedure

1. Process the first day's file.

2. Process the second day's file.
3. Process the third day's file.
4. Display the contents of the log.
5. Compare the results with the expected records.

When the original program is used, write mode overwrites the previous content each time. Therefore, after the third execution, only the records from day 3 remain.

The missing newline characters also cause records to be joined together.

Table 4.2 – Evidence of the Original Defects

Test	Operation	Expected behaviour	Original program behaviour
1	Process day 1	Store day 1 books	Stores day 1 books
2	Process day 2	Preserve day 1 and add day 2	Overwrites day 1
3	Process day 3	Preserve day 1 and day 2 and add day 3	Overwrites day 2
4	Display log	Each book on a separate line	Book names may be joined
5	Process missing file	Record the error permanently	Only prints a message

Task 4.3 – Repair the Program

The corrected implementation uses:

- Append mode (a) to preserve previous records.
- Newline characters to separate book names.
- with statements to close files automatically.
- An error log to record file-processing failures.
- UTF-8 encoding for consistent text handling.

Corrected Python Program

```
from pathlib import Path


BASE_DIR = Path(__file__).parent
ERROR_LOG = BASE_DIR / "error.log"
RETURNS_LOG = BASE_DIR / "returns.log"


def save_returns(books):
    with open(
        RETURNS_LOG,
        "a",
        encoding="utf-8"
    ) as log:

        for book in books:
            book = book.strip()

            if book:
                log.write(book + "\n")


def process(day_file):
    try:
        with open(
            day_file,
            "r",
            encoding="utf-8"
        ) as file:
            books = file.readlines()
```

```
        save_returns(books)

    except FileNotFoundError as error:
        message = (
            f"{day_file}: FileNotFoundError: {error}\n"
        )

        with open(
            ERROR_LOG,
            "a",
            encoding="utf-8"
        ) as log:
            log.write(message)

        print("Error recorded:", message.strip())

    except PermissionError as error:
        with open(
            ERROR_LOG,
            "a",
            encoding="utf-8"
        ) as log:
            log.write(
                f"{day_file}: PermissionError: {error}\n"
            )

        print("Permission error recorded.")


if __name__ == "__main__":
    process(BASE_DIR / "day1.txt")
    process(BASE_DIR / "day2.txt")
```

```
process(BASE_DIR / "day3.txt")
process(BASE_DIR / "missing_day.txt")

print("Returns log:")

with open(
    RETURNS_LOG,
    "r",
    encoding="utf-8"
) as file:
    print(file.read())

print("Error log:")

if ERROR_LOG.exists():
    with open(
        ERROR_LOG,
        "r",
        encoding="utf-8"
    ) as file:
        print(file.read())
```

Actual Execution Output

Error recorded: /workspaces/CSA0823---Python-Programing/C04 Assignment 4/Task04/missing_day.txt: FileNotFoundError: [Errno 2] No such file or directory: '/workspaces/CSA0823---Python-Programing/C04 Assignment 4/Task04/missing_day.txt'

Returns log:

The Alchemist

Atomic Habits

White Nights

Metamorphosis

Ikigai

Wings of Fire

Error log:

/workspaces/CSA0823---Python-Programing/C04 Assignment 4/Task04/missing_day.txt: FileNotFoundError: [Errno 2] No such file or directory: '/workspaces/CSA0823---Python-Programing/C04 Assignment 4/Task04/missing_day.txt'

Table 4.3 – Corrected Program Results

S.No.	Test case	Result
1	Process day1.txt	Successfully stored two books
2	Process day2.txt	Preserved earlier records and added two books
3	Process day3.txt	Preserved earlier records and added two books
4	Process missing_day.txt	FileNotFoundError detected
5	Error logging	Missing-file error stored in error.log
6	Log verification	All six books displayed on separate lines

Generated Files

File	Purpose
returns.log	Stores returned book names from all processed days
error.log	Stores file-processing errors

Task 4.4 – Prevent Recurrence

The following two file-handling rules would have prevented the defects:

Rule 1: Use the correct file mode for the operation. Use append mode (a) when existing data must be preserved, and include newline characters when separate records must be stored on separate lines.

Rule 2: Always use a with statement when opening files. It automatically closes the file after the operation, even when an exception occurs.

Alternative Concept and Complexity Analysis

An alternative to using with is to open and close files manually using try and finally.

Example:

```
file = None

try:
    file = open("returns.log", "a", encoding="utf-8")
    file.write("The Alchemist\n")

finally:
    if file is not None:
        file.close()
```

The finally block ensures that the file is closed even if an exception occurs.

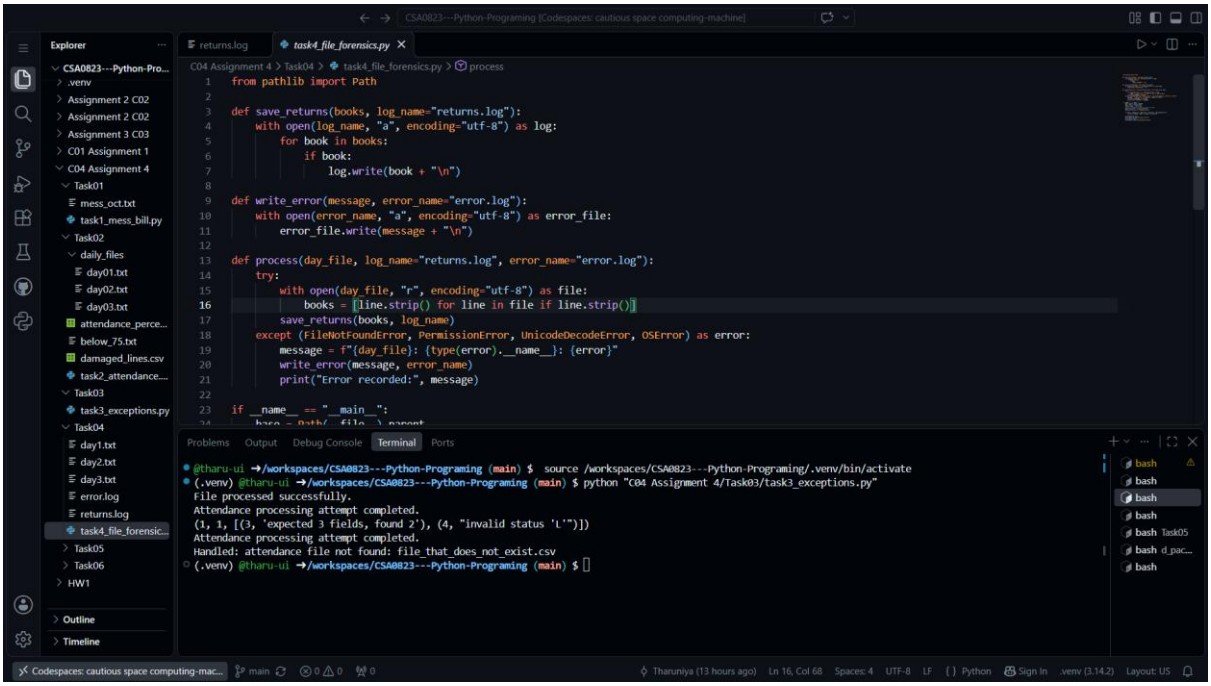
Table 4.4 – Comparison of File-Closing Methods

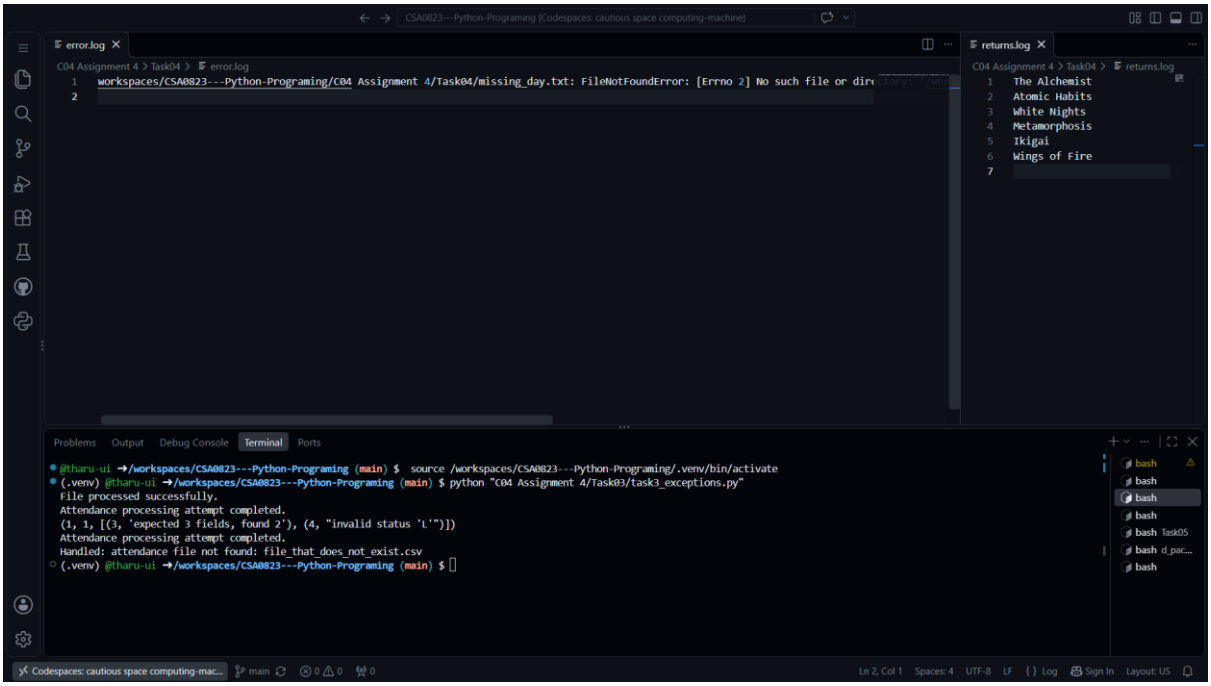
Comparison	**with statement**	**Manual try/finally**
File opening	Uses context manager	Uses open()
File closing	Automatic	Must be written explicitly
Error handling	Automatically closes file	Closes file through finally
Code complexity	Simple and readable	More statements required

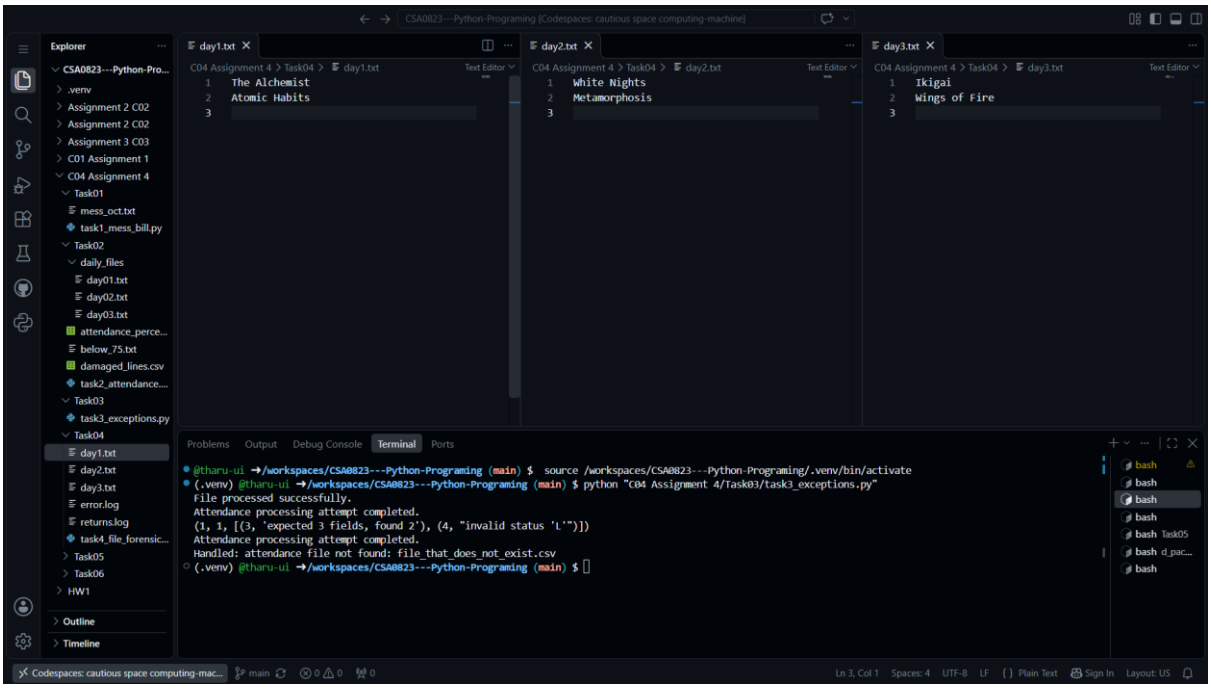
Time complexity	O(n) for n records	O(n) for n records
Space complexity	O(1) extra space for line-by-line writing	O(1) extra space for line-by-line writing
Risk of resource leakage	Low	Low if finally is correctly implemented

In the corrected version, the input file is closed before the output file is opened. During writing, only the output file remains open. In the manual approach, the same resource behaviour is achieved only if the finally block is correctly implemented.

Conclusion: Append mode, newline characters, context managers and error logging ensure reliable storage of returned-book records.




error.log
C04 Assignment 4 > Task04 > error.log
workspaces/CSA0823---Python-Programing/C04 Assignment 4/Task04/missing_day.txt: FileNotFoundError: [Errno 2] No such file or dir
returns.log
C04 Assignment 4 > Task04 > returns.log
The Alchemist
Atomic Habits
White Nights
Metamorphosis
Ikigai
Wings of Fire
Problems Output Debug Console Terminal Ports
@tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ source /workspaces/CSA0823---Python-Programing/.venv/bin/activate
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ python "C04 Assignment 4/Task03/task3_exceptions.py"
File processed successfully.
Attendance processing attempt completed.
(1, 1, [(3, 'expected 3 fields, found 2'), (4, "invalid status 'L'")])
Attendance processing attempt completed.
Handled: attendance file not found: file_that_does_not_exist.csv
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $


Explorer
CSA0823---Python-Pro...
.venv
Assignment 2 C02
Assignment 2 C02
Assignment 3 C03
C01 Assignment 1
C04 Assignment 4
Task01
mess_oct.txt
task1_mess_bill.py
Task02
daily_files
day01.txt
day02.txt
day03.txt
attendance_perce...
below_75.txt
damaged_lines.csv
task2_attendance....
Task03
task3_exceptions.py
Task04
day1.txt
day2.txt
day3.txt
error.log
returns.log
task4_file_forensic...
Task05
Task06
HW1
Outline
Timeline
day1.txt
C04 Assignment 4 > Task04 > day1.txt
The Alchemist
Atomic Habits
day2.txt
C04 Assignment 4 > Task04 > day2.txt
White Nights
Metamorphosis
day3.txt
C04 Assignment 4 > Task04 > day3.txt
Ikigai
Wings of Fire
Problems Output Debug Console Terminal Ports
@tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ source /workspaces/CSA0823---Python-Programing/.venv/bin/activate
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ python "C04 Assignment 4/Task03/task3_exceptions.py"
File processed successfully.
Attendance processing attempt completed.
(1, 1, [(3, 'expected 3 fields, found 2'), (4, "invalid status 'L'")])
Attendance processing attempt completed.
Handled: attendance file not found: file_that_does_not_exist.csv
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $

TASK 5 – THE SAME UTILITY ORGANISED FOUR WAYS

Bloom's Level: K5 – Evaluate
Marks: 7

Task 5.1 – Four Organisations

The attendance processor consists of three main groups of functions:

1. Reading attendance files.
2. Computing attendance statistics.
3. Writing reports.

The same functionality can be organised in four different ways.

A. One Script

All functions are placed in a single Python file.

Structure:

a_single_script.py

sample_days/

day1.txt

day2.txt

day3.txt

day4.txt

Explanation:

The reading, calculation and reporting functions are defined in one script. This structure is simple and suitable for small programs.

B. One Module Imported by a Main Script

The utility functions are placed in a separate module, while the main program imports and uses them.

Structure:

b_module/

attendance_utils.py

main.py

Explanation:

The attendance_utils.py module contains reusable attendance-processing functions. The main.py file imports the module and executes the program.

Example:

import attendance_utils

Call the required utility functions

This structure separates program logic from program execution.

C. Three Modules in One Folder

The program is divided into three separate modules based on responsibility.

Structure:

c_three_modules/

reader.py

stats.py

report.py

main.py

Explanation:

- reader.py reads attendance files.
- stats.py calculates attendance percentages and consecutive absences.
- report.py writes the generated reports.
- main.py coordinates the execution.

This arrangement improves modularity and makes individual functions easier to test.

D. Package Organisation

A package is created using a directory named attendance, containing an __init__.py file and three sub-modules.

Structure:

d_package/

attendance/

__init__.py

reader.py

stats.py

report.py

main.py

Explanation:

The attendance directory acts as a Python package.

- reader.py contains file-reading functions.
- stats.py contains attendance calculations.
- report.py contains report-generation functions.
- __init__.py identifies and initialises the package.
- main.py imports and uses the package.

This structure is suitable for larger applications because it organises related modules under a common package.

Table 5.1 – Comparison of Four Organisations

Feature	One script	One module	Three modules	Package
Number of logical code units	One	Two	Four	Four
Organisation	All functions together	Utility and main separated	Responsibilities separated	Modules grouped inside a package
Reusability	Low	Moderate	High	High
Readability	Decreases as program grows	Better separation	Clear responsibility-based separation	Structured and scalable
Testing	Difficult for individual functions	Utility module can be tested	Individual modules can be tested	Each package module can be tested
Maintenance	More difficult for large programs	Easier	Easier	Suitable for larger applications
Naming conflicts	More likely in one namespace	Reduced	Reduced through module names	Better namespace organisation

Task 5.2 – Change the Requirement

The department now requires an additional report containing the register numbers of students who were absent for three consecutive days.

The new functionality must identify three consecutive A records for the same student.

Logic for Consecutive Absences

For every student:

1. Read the attendance status for each day in chronological order.
2. Maintain a consecutive-absence counter.
3. If the status is A, increment the counter.
4. If the status is P, reset the counter to zero.
5. If the counter reaches three, add the student to the consecutive-absence report.

Pseudocode

```
BEGIN

  INITIALIZE consecutive_absence_count = 0

  FOR each attendance status of a student DO

    IF status = A THEN
      INCREMENT consecutive_absence_count
    ELSE
      SET consecutive_absence_count = 0
    END IF

    IF consecutive_absence_count = 3 THEN
      ADD student register number to report
    END IF

  END FOR

END
```

Table 5.2 – Consecutive Absence Example

Student	Day 1	Day 2	Day 3	Day 4	Consecutive absence result
192311001	P	P	A	P	Not identified
192311002	A	A	A	A	Identified

Actual Execution Results

All four implementations generated the same report.

192311001: 75.00%

192311002: 0.00%

Absent three consecutive days: 192311002

Table 5.3 – Files Involved in the New Requirement

Organisation	Files involved	Reason
One script	a_single_script.py	The new logic and report generation are in the same file.
One module	attendance_utils.py, main.py	Utility logic and the main program must use the new report.
Three modules	stats.py, report.py, main.py	Statistics, report writing and program coordination are involved.
Package	attendance/stats.py, attendance/report.py, main.py	The package calculation and reporting modules are updated and the main program invokes the new report.

The number of files that need to change depends on where the new functionality is placed. In the implementations used for this assignment, the new report is supported in all four organisations.

Task 5.3 – Compare the Organisations

Readability

A single script is easy to understand when the program is small. As the number of functions increases, locating and modifying specific operations becomes more difficult.

A module-based structure separates utility functions from the main program. Three modules provide clearer separation of responsibilities. A package offers a structured arrangement suitable for larger projects.

Reusability

Functions in separate modules can be imported into other programs. A package allows related modules to be grouped and reused together.

Testing

In a single script, testing individual functions may be less convenient. In modular programs, each module can be tested separately. This helps identify errors more quickly.

Naming Conflicts

A single script contains all functions and variables in the same module namespace. Separate modules provide their own namespaces and reduce the possibility of naming conflicts.

Table 5.4 – Overall Evaluation

Evaluation factor	**One script**	**One module**	**Three modules**	**Package**
Readability	Suitable for small programs	Improved	High	High
Reuse	Limited	Moderate	High	High
Testing	Less convenient	Easier	Easier	Easier
Naming conflicts	More likely	Reduced	Reduced	Reduced
Scalability	Low	Moderate	High	Very high

Maintenance	Difficult as code grows	Improved	Good	Good

Difference Between Import Statements

1. import attendance.stats

Imports the stats sub-module from the attendance package. Functions are accessed using the module name.

Example:

import attendance.stats

attendance.stats.percentage()

2. from attendance import stats

Imports the stats sub-module from the attendance package directly.

Example:

from attendance import stats

stats.percentage()

3. from attendance.stats import percentage

Imports only the percentage function from the stats module.

Example:

from attendance.stats import percentage

percentage()

Table 5.5 – Comparison of Import Statements

Import statement	**What is imported?**	**Function access**
import attendance.stats	The module through its package path	attendance.stats.percentage()

from attendance import stats	The stats module	stats.percentage()
from attendance.stats import percentage	The specific function	percentage()

Alternative Concept and Complexity Analysis

If all functions remained in one script instead of being organised into a package, the program would still execute correctly if all functions were defined properly. However, the code would become harder to maintain as the application grew.

Python loads imported modules and stores them in sys.modules. When the same module is imported again during the same interpreter session, Python normally reuses the cached module rather than loading and executing it again.

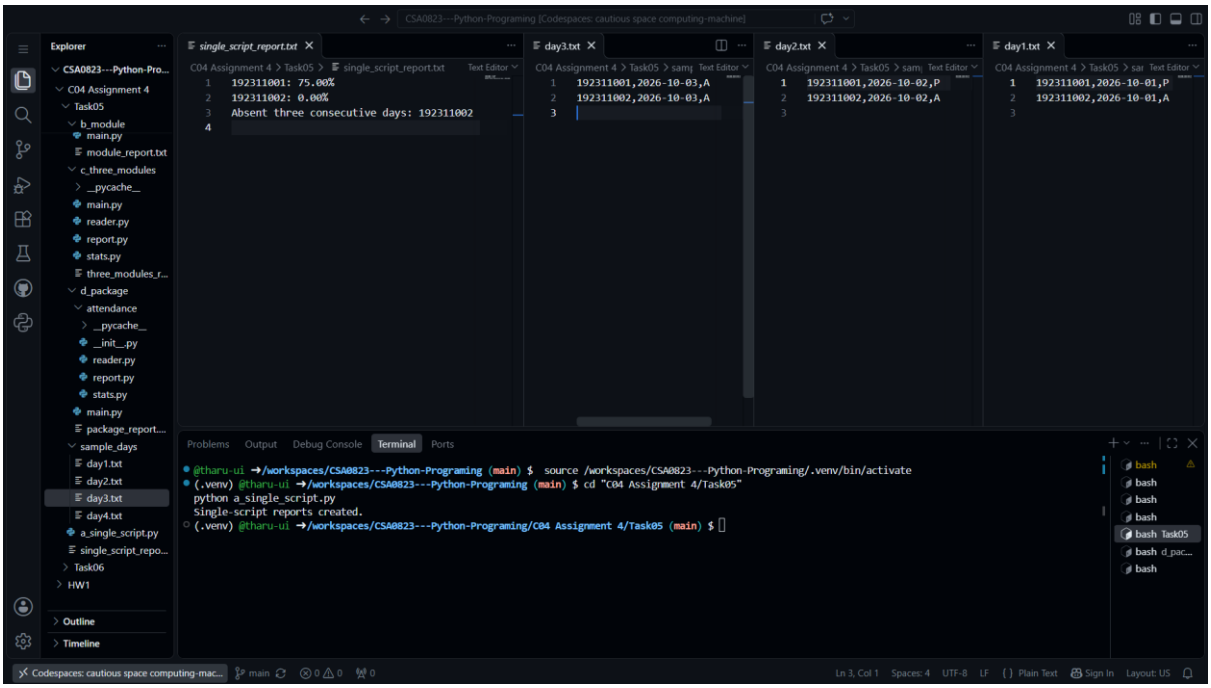
Table 5.6 – Package Import and Maintenance Comparison

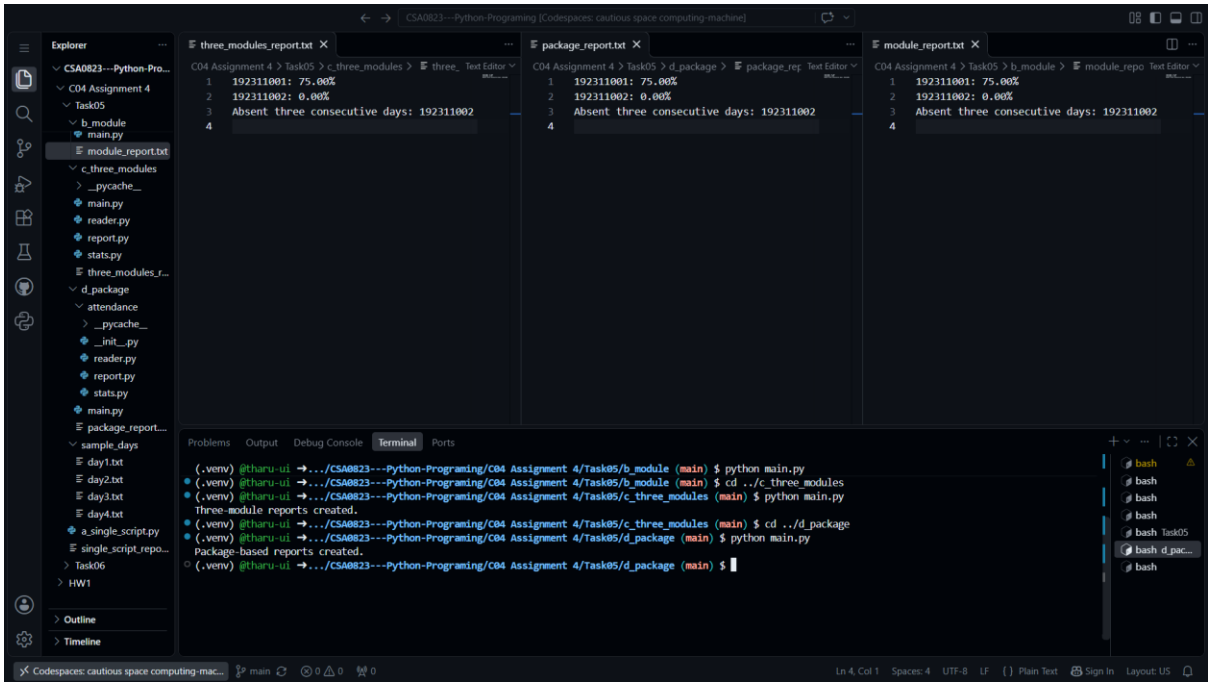
Comparison	**Single script**	**Package**
Initial organisation	All functions in one file	Functions separated into modules
Import operation	Not required for internal functions	Modules imported when needed
Module caching	Not applicable to internal functions	Imported modules are cached in sys.modules
Import memory	No separate imported utility modules	Imported module objects occupy memory
Finding a function	Search through the entire script	Locate the relevant module
Changing a function	May affect a large file	Usually limited to the responsible module
Maintenance effort	Increases as code grows	Reduced through separation of responsibilities

Time complexity: The attendance calculation still requires processing the input records. Organising functions into modules does not change the fundamental O(n) processing complexity.

Space complexity: A package requires memory for the imported module objects and their definitions. The additional memory is related to the imported modules; it does not change the basic attendance data-processing requirements.

Conclusion: A package provides better organisation, reusability and maintainability than a single script. It is more suitable for applications that are expected to grow.




CSA0823---Python-Programing [Codespaces: cautious space computing-machine]
Explorer
CSA0823---Python-Pro...
C04 Assignment 4
Task05
b_module
main.py
module_report.txt
c_three_modules
__pycache__
main.py
reader.py
report.py
stats.py
three_modules_r...
d_package
attendance
__pycache__
__init__.py
reader.py
report.py
stats.py
main.py
package_report....
sample_days
day1.txt
day2.txt
day3.txt
day4.txt
a_single_script.py
single_script_repo...
Task06
HW1
Outline
Timeline
three_modules_report.txt
C04 Assignment 4 > Task05 > c_three_modules > three_ Text Editor
1 192311001: 75.00%
2 192311002: 0.00%
3 Absent three consecutive days: 192311002
4
package_report.txt
C04 Assignment 4 > Task05 > d_package > package_rep Text Editor
1 192311001: 75.00%
2 192311002: 0.00%
3 Absent three consecutive days: 192311002
4
module_report.txt
C04 Assignment 4 > Task05 > b_module > module_repo Text Editor
1 192311001: 75.00%
2 192311002: 0.00%
3 Absent three consecutive days: 192311002
4
Problems
Output
Debug Console
Terminal
Ports
(.venv) @tharu-ui ➜.../CSA0823---Python-Programing/C04 Assignment 4/Task05/b_module (main) $ python main.py
(.venv) @tharu-ui ➜.../CSA0823---Python-Programing/C04 Assignment 4/Task05/b_module (main) $ cd ../c_three_modules
(.venv) @tharu-ui ➜.../CSA0823---Python-Programing/C04 Assignment 4/Task05/c_three_modules (main) $ python main.py
Three-module reports created.
(.venv) @tharu-ui ➜.../CSA0823---Python-Programing/C04 Assignment 4/Task05/c_three_modules (main) $ cd ../d_package
(.venv) @tharu-ui ➜.../CSA0823---Python-Programing/C04 Assignment 4/Task05/d_package (main) $ python main.py
Package-based reports created.
(.venv) @tharu-ui ➜.../CSA0823---Python-Programing/C04 Assignment 4/Task05/d_package (main) $
bash
bash
bash
bash
bash Task05
bash d_pac...
bash
Codespaces: cautious space computing-mac...
main
Ln 4, Col 1
Spaces: 4
UTF-8
LF
Plain Text
Sign In
Layout: US

TASK 6 – DESIGN CHALLENGE: COLLEGE FEST REGISTRATION DESK

Bloom's Level: K5 – Evaluate
Marks: 13

Stage 1 – Interpret the Problem

The college cultural fest receives student registrations through text files submitted by five departments. Each record contains the student's register number, name, department, e-mail address and selected events.

The system must process all department files and generate the following outputs:

1. A master registration list containing valid and unique registrations.
2. Separate participant lists for each event.
3. A summary report containing the total number of registrations and event-wise participant counts.
4. An error log containing duplicate, invalid and damaged records.

The program must continue processing the remaining records even when it encounters invalid data or a missing department file.

Input File Format

Each registration record follows this format:

RegisterNumber,Name,Department,Email,Event1;Event2

Example:

192311001,Anu,CSE,anu@example.com,Music;Dance

Table 6.1 – Input Data Description

S.No.	Field	Description	Example
1	Register number	Unique student identifier	192311001
2	Name	Student's name	Anu
3	Department	Student's department	CSE

4	E-mail	Student's e-mail address	anu@example.com
5	Events	Semicolon-separated list of selected events	Music;Dance

Table 6.2 – Required Output Files

S.No.	Output file	Purpose
1	master_list.csv	Stores all valid and unique registrations
2	Event-wise participant files	Stores participants registered for each event
3	summary_report.txt	Displays the total registrations and event-wise participant counts
4	error.log	Records duplicate, invalid and damaged records

Table 6.3 – Functional Requirements

S.No.	Requirement	Testable condition
1	Read department files	All available department files must be processed.
2	Validate record structure	Each record must contain exactly five comma-separated fields.
3	Validate register number	Register numbers must not be empty.
4	Validate e-mail address	E-mail must contain a local part, an @ symbol and a valid-looking domain.
5	Detect duplicates	A register number must be accepted only once.
6	Reject damaged records	Records with missing or extra fields must be logged.

7	Generate master list	All valid unique records must be written to a CSV file.
8	Generate event lists	Every valid participant must appear in the corresponding event file.
9	Generate summary	Total registrations and event-wise participant counts must be displayed.
10	Maintain error log	Invalid records and processing errors must be recorded.
11	Handle missing files	A missing department file must be logged without terminating the entire process.

Stage 2 – Propose the Processing Flow

The registration-processing system follows a sequence of reading, validation, duplicate checking and report generation.

Processing Flow

START

|

v

Identify the five department files

|

v

Initialize registration data structures

|

v

Read the next available department file

|

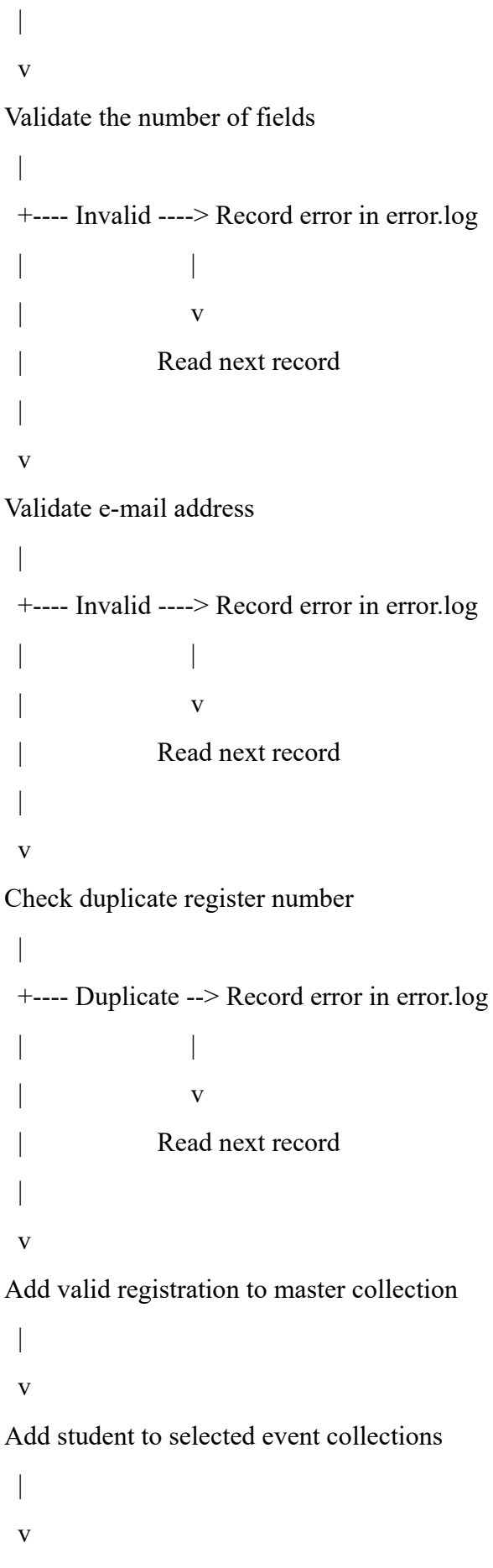
v

Read each record line by line

|

v

Split the record into fields


Validate the number of fields
Invalid
Record error in error.log
Read next record
Validate e-mail address
Invalid
Record error in error.log
Read next record
Check duplicate register number
Duplicate
Record error in error.log
Read next record
Add valid registration to master collection
Add student to selected event collections

Are more records or files available?

|

+---- Yes --------> Continue processing

|

No

|

v

Generate master_list.csv

|

v

Generate event-wise participant files

|

v

Generate summary_report.txt

|

v

Save error.log

|

v

Display processing summary

|

v

END

Explanation

The program processes each department file separately. Every record is validated before it is accepted. Records containing an incorrect number of fields, invalid e-mail addresses or duplicate register numbers are rejected and recorded in the error log.

Valid records are stored in a master collection. Separate event-wise collections are maintained to generate participant lists. After all available files have been processed, the program generates the master registration list, event-wise files, summary report and error log.

Stage 3 – Identify the Critical Design Decisions

3.1 Duplicate Detection Using a Set

A Python set is used to store register numbers that have already been accepted.

Example:

```
seen_register_numbers = set()
```

Before accepting a registration, the program checks whether its register number already exists in the set.

```
if register_number in seen_register_numbers:
    # Reject duplicate record
    pass
else:
    seen_register_numbers.add(register_number)
```

A set uses hash-based lookup and provides average O(1) membership checking. Therefore, it is efficient for detecting duplicate registrations in large datasets.

3.2 E-mail Validation Using String Methods

Basic e-mail validation can be performed using Python string methods.

The validation conditions include:

- The e-mail address must not be empty.
- It must contain exactly one @ character.
- The local part must not be empty.
- The domain must contain a dot.
- The domain must not begin or end with a dot.
- Spaces must not be present.

Example:

```
def valid_email(email):
    email = email.strip()
```

```
    if not email:
        return False

    if email.count("@") != 1:
        return False

    if " " in email:
        return False

    local, domain = email.split("@")

    if not local or not domain:
        return False

    if "." not in domain:
        return False

    if domain.startswith(".") or domain.endswith("."):
        return False

    return True
```

This method performs basic e-mail validation suitable for the assignment. It does not implement every rule of the complete e-mail address standard.

3.3 Exception Handling

The program must handle file-related errors and invalid records without stopping the entire registration process.

Table 6.4 – Exception Handling Design

S.No.	Exception or validation condition	Where it may occur	Handling method
1	FileNotFoundError	A department file is missing	Record the missing file and continue with the remaining files.
2	PermissionError	The program cannot access a file	Record the permission error.
3	UnicodeDecodeError	The file contains data that cannot be decoded using the selected encoding	Record the encoding error.
4	Invalid record structure	A record contains an incorrect number of fields	Reject the record and log the reason.
5	Invalid e-mail	E-mail validation fails	Reject the record and log the reason.
6	Duplicate registration	Register number already exists in the set	Reject the duplicate and log the reason.
7	OSError	Another file-system operation fails	Record the file-processing error.

The processing of one invalid record should not terminate the complete registration process. File-level errors should also be handled separately so that the remaining department files can still be processed.

Stage 4 – Design the Package

The registration system is organised using a Python package named registration.

4.1 Package Structure

```
fest_registration/
│
├── main.py
│
├── registration/
│   ├── __init__.py
│   ├── reader.py
│   ├── validator.py
│   ├── processor.py
│   └── report.py
│
├── department_files/
│   ├── department1.txt
│   ├── department2.txt
│   ├── department3.txt
│   ├── department4.txt
│   └── department5.txt
│
└── output/
    ├── master_list.csv
    ├── Music_participants.txt
    ├── Dance_participants.txt
    ├── Drama_participants.txt
```

├── Quiz_participants.txt

├── Photography_participants.txt

├── summary_report.txt

└── error.log

Table 6.5 – Package Module Responsibilities

S.No.	Module	Responsibility	Main operations
1	reader.py	Reads department files	Reads records and tracks their source information.
2	validator.py	Validates registration data	Checks record fields and e-mail addresses.
3	processor.py	Processes registrations	Detects duplicates and organises valid records.
4	report.py	Generates output files	Writes the master list, event lists and summary report.
5	__init__.py	Initialises the package	Makes package-level imports available.
6	main.py	Controls execution	Calls the package functions in the required order.

4.2 Pseudocode – Reading Department Files

BEGIN

GET list of department files

FOR each department file DO

TRY

```
        OPEN file in READ mode

        SET line_number = 0

        FOR each line in file DO

            INCREMENT line_number
            STORE line and source information

        END FOR

        CLOSE file

    EXCEPT FileNotFoundError
        RECORD missing file error
        CONTINUE with next file

    EXCEPT PermissionError
        RECORD permission error
        CONTINUE with next file

  END FOR

END
```

4.3 Pseudocode – Validating and Processing Records

```
BEGIN

  INITIALIZE valid_registrations
  INITIALIZE seen_register_numbers
  INITIALIZE event_participants
```

```
INITIALIZE error_records

FOR each input record DO

    SPLIT record using comma

    IF number of fields is not equal to 5 THEN
        RECORD damaged record
        CONTINUE
    END IF

    EXTRACT register number, name, department,
            e-mail and events

    VALIDATE e-mail address

    IF e-mail is invalid THEN
        RECORD invalid e-mail
        CONTINUE
    END IF

    IF register number exists in seen_register_numbers THEN
        RECORD duplicate registration
        CONTINUE
    END IF

    ADD register number to seen_register_numbers

    ADD registration to valid_registrations

    SPLIT selected events using semicolon
```

```
        FOR each event DO
            ADD student to corresponding event list
        END FOR

    END FOR

END
```

4.4 Pseudocode – Generating Reports

```
BEGIN

    OPEN master_list.csv in WRITE mode

    WRITE column headings

    FOR each valid registration DO
        WRITE registration details
    END FOR

    CLOSE master_list.csv

    FOR each event DO

        OPEN corresponding participant file in WRITE mode

        FOR each participant in event list DO
            WRITE participant details
        END FOR

        CLOSE participant file

    END FOR
```

```
    OPEN summary_report.txt in WRITE mode

    WRITE total valid registrations
    WRITE total events

    FOR each event DO
        WRITE event name and participant count
    END FOR

    CLOSE summary_report.txt

    WRITE all rejected-record details to error.log

END
```

Stage 5 – Implement and Demonstrate

The registration package was implemented and executed using five department files.

The input data included:

- Valid student registrations.
- A duplicate registration.
- An invalid e-mail address.
- A damaged record containing an incorrect number of fields.

The program generated a master registration list, separate event-wise participant files, a summary report and an error log.

5.1 Actual Execution Summary

The recorded execution displayed the following output:

Input records read: 8

Valid unique registrations: 6

Output folder: /workspaces/CSA0823---Python-Programing/C04 Assignment 4/Task06/fest_registration/output

Error log: /workspaces/CSA0823---Python-Programing/C04 Assignment 4/Task06/fest_registration/output/error.log

Table 6.6 – Recorded Execution Results

Description	**Actual result**
Input records read, as reported by the program	8
Valid unique registrations	6
Total event categories	5

The error log contains three rejected-record entries: one damaged record, one duplicate registration and one invalid e-mail address. The input counter and rejected-record categories should be checked against the source files before final submission to ensure that every input line is counted consistently.

5.2 Actual Master Registration List

The generated master_list.csv contained the following records:

Register Number,Name,Department,Email,Events

192311001,Anu,CSE,anu@example.com,"Music;Dance"

192311002,Bala,CSE,bala@example.com,"Drama"

192311003,Charu,ECE,charu@example.com,"Quiz;Dance"

192311005,Esha,IT,esha@example.com,"Photography"

192311006,Fathima,MECH,fathima@example.com,"Drama;Quiz"

192311007,Gayathri,CIVIL,gaya@example.com,"Music"

Table 6.7 – Accepted Registrations

S.No.	Register Number	Name	Department	E-mail	Selected events
1	192311001	Anu	CSE	anu@example.com	Music, Dance
2	192311002	Bala	CSE	bala@example.com	Drama
3	192311003	Charu	ECE	charu@example.com	Quiz, Dance
4	192311005	Esha	IT	esha@example.com	Photography
5	192311006	Fathima	MECH	fathima@example.com	Drama, Quiz
6	192311007	Gayathri	CIVIL	gaya@example.com	Music

The master list contains six valid unique registrations. The duplicate registration and records with invalid data were excluded.

5.3 Actual Summary Report

The generated summary_report.txt contained:

Total valid unique registrations: 6

Total events: 5

Dance: 2 participants

Drama: 2 participants

Music: 2 participants

Photography: 1 participants

Quiz: 2 participants

Table 6.8 – Event-wise Participant Counts

S.No.	Event	Number of participants
1	Dance	2
2	Drama	2
3	Music	2
4	Photography	1
5	Quiz	2
	Total event registrations	**9**

Note: The total event registrations are 9 because a student may participate in more than one event. The number of unique registered students remains 6.

5.4 Actual Error Log

The generated error.log contained:

department3.txt, line 2: damaged record: expected 5 comma-separated fields

CSE, line 0: duplicate registration 192311001

EEE, line 0: invalid e-mail 'deepa.example.com'

Table 6.9 – Error Log Analysis

S.No.	Error type	Source information recorded	Reason	Action
1	Damaged record	department3.txt, line 2	Expected 5 comma-separated fields	Rejected and logged
2	Duplicate registration	CSE, line 0	Register number 192311001 already exists	Rejected and logged

3	Invalid e-mail	EEE, line 0	Missing @ symbol	Rejected and logged

Implementation observation: The current error log records line 0 for the duplicate and invalid e-mail entries. This indicates that the implementation does not preserve the actual source line number for those two errors. The reader and processor should pass the original file name and line number to the error logger. The final implementation should record the actual line numbers rather than using zero.

5.5 Missing Department File Test

The five supplied department files were present during the recorded execution. Therefore, a missing department file was not demonstrated in this actual run.

To test this condition, one department file can temporarily be renamed or moved before executing the program again.

Expected behaviour:

1. The program attempts to open the missing department file.
2. Python raises FileNotFoundError.
3. The exception is caught by the file-processing code.
4. The missing file is recorded in the error log.
5. The program continues processing the remaining department files.

This test verifies that a missing input file does not terminate the entire registration process.

Stage 6 – Evaluate an Alternative Approach

An alternative duplicate-detection method is to search through a list of previously accepted register numbers before accepting a new registration.

Example:

register_numbers = []

if register_number in register_numbers:

```
    # Duplicate found
    pass
else:
    register_numbers.append(register_number)
```

This approach checks the list sequentially for every new registration.

Table 6.10 – Comparison of Duplicate-Detection Methods

Comparison	**Set-based method**	**List-search method**
Data structure	Set	List
Duplicate checking	Hash-based lookup	Sequential search
Average lookup time	O(1)	O(n)
Total time for n registrations	O(n) average	$O(n^2)$
Extra space	O(n)	O(n)
Duplicate detection	Efficient	Slower for large datasets
Suitability	Recommended	Suitable for small datasets

Time Complexity Explanation

Set-based method:

Each register number is checked against a set. Average membership checking takes O(1) time. Therefore, for n registrations, the average total time complexity is:

[O(n)]

List-search method:

Each new register number may require checking all previously accepted register numbers. In the worst case, the number of comparisons increases as the list grows.

The total number of comparisons is:

[1+2+3+\cdots+(n-1)]

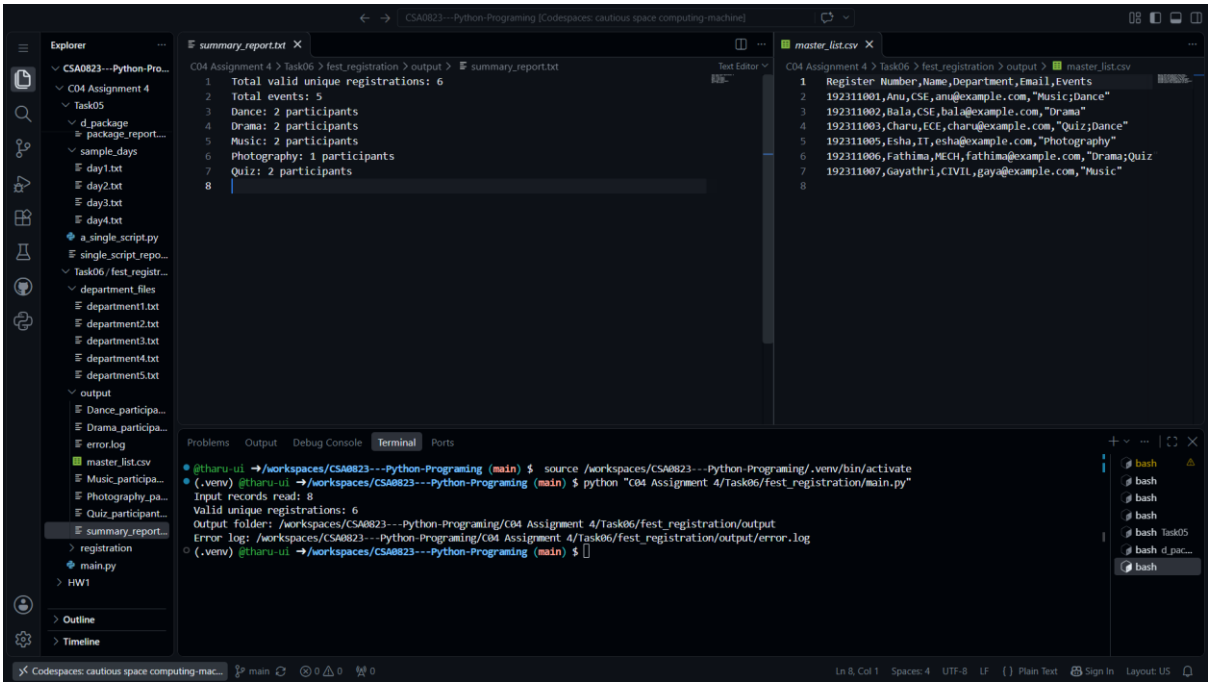
Therefore, the time complexity is:

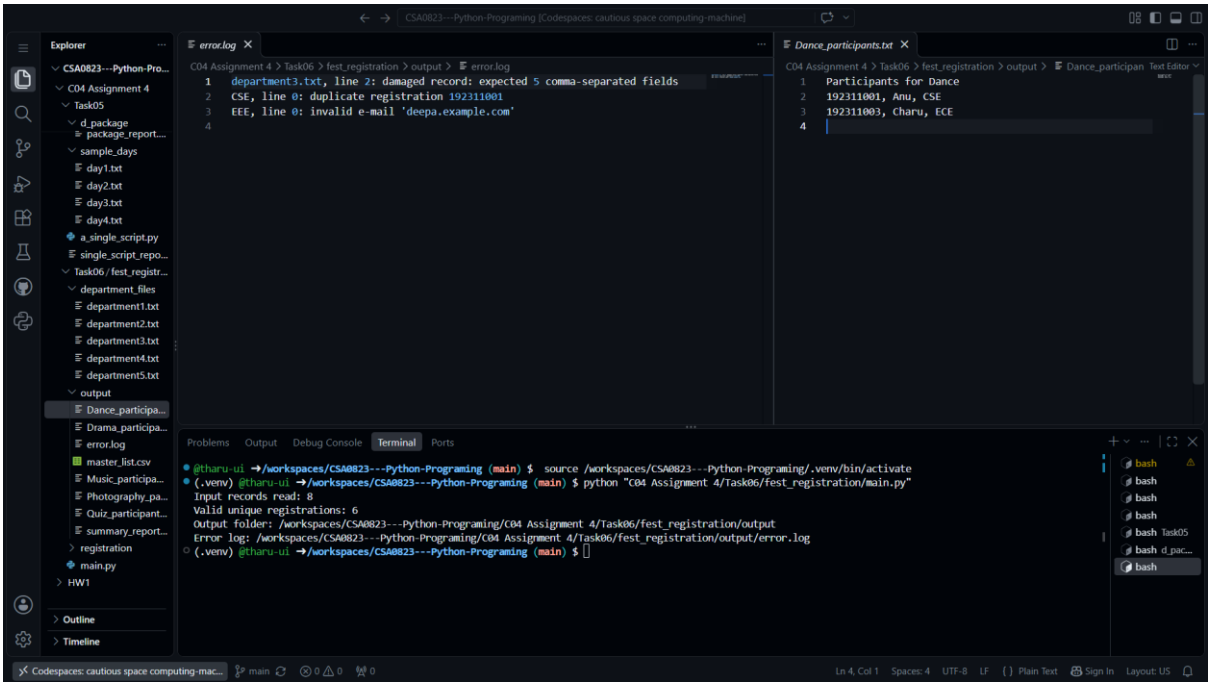
[O(n^2)]

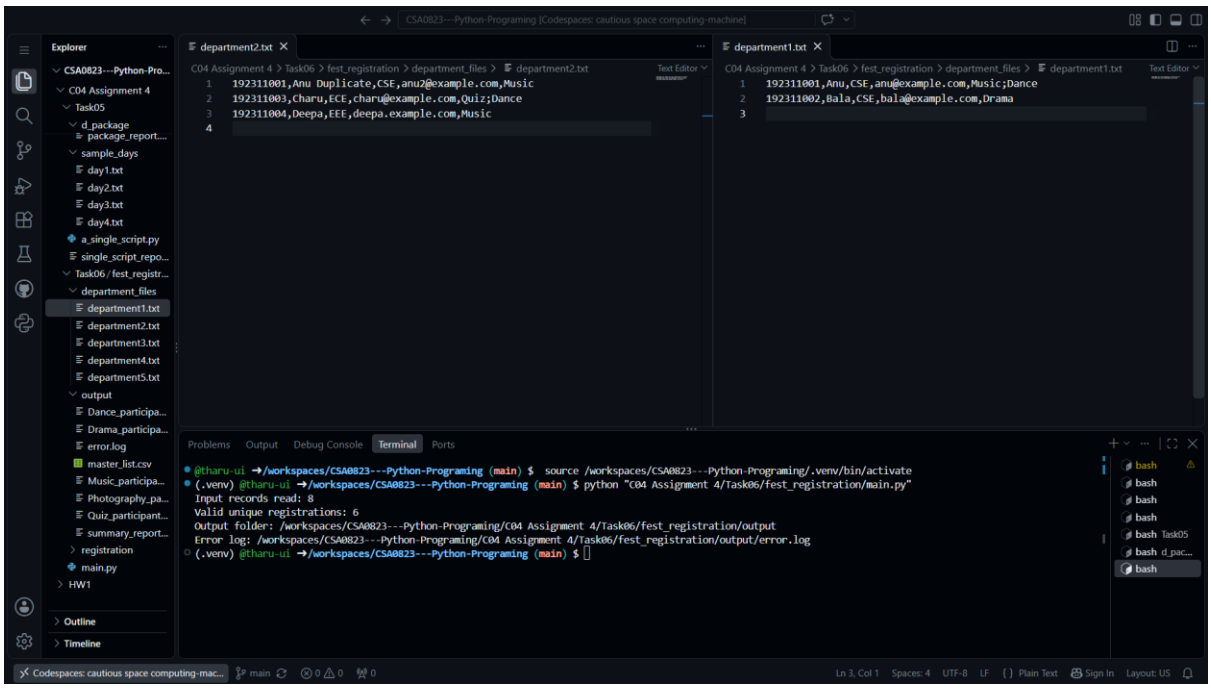
Both methods require O(n) space to store accepted register numbers.

Conclusion

A set is more efficient for duplicate detection because it provides average constant-time membership checking. A list-search approach becomes slower as the number of registrations increases. Therefore, a set is more suitable for processing large registration files.




error.log
1 department3.txt, line 2: damaged record: expected 5 comma-separated fields
2 CSE, line 0: duplicate registration 192311001
3 EEE, line 0: invalid e-mail 'deepa.example.com'
Dance_participants.txt
1 Participants for Dance
2 192311001, Anu, CSE
3 192311003, Charu, ECE
Problems Output Debug Console Terminal Ports
@tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ source /workspaces/CSA0823---Python-Programing/.venv/bin/activate
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ python "C04 Assignment 4/Task06/fest_registration/main.py"
Input records read: 8
Valid unique registrations: 6
Output folder: /workspaces/CSA0823---Python-Programing/C04 Assignment 4/Task06/fest_registration/output
Error log: /workspaces/CSA0823---Python-Programing/C04 Assignment 4/Task06/fest_registration/output/error.log
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $


department2.txt
1 192311001,Anu Duplicate,CSE,anu2@example.com,Music
2 192311003,Charu,ECE,charu@example.com,Quiz;Dance
3 192311004,Deepa,EEE,deepa.example.com,Music
department1.txt
1 192311001,Anu,CSE,anu@example.com,Music;Dance
2 192311002,Bala,CSE,bala@example.com,Drama
Problems Output Debug Console Terminal Ports
@tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ source /workspaces/CSA0823---Python-Programing/.venv/bin/activate
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $ python "C04 Assignment 4/Task06/fest_registration/main.py"
Input records read: 8
Valid unique registrations: 6
Output folder: /workspaces/CSA0823---Python-Programing/C04 Assignment 4/Task06/fest_registration/output
Error log: /workspaces/CSA0823---Python-Programing/C04 Assignment 4/Task06/fest_registration/output/error.log
(.venv) @tharu-ui ➜/workspaces/CSA0823---Python-Programing (main) $

APPENDIX – EXECUTION EVIDENCE

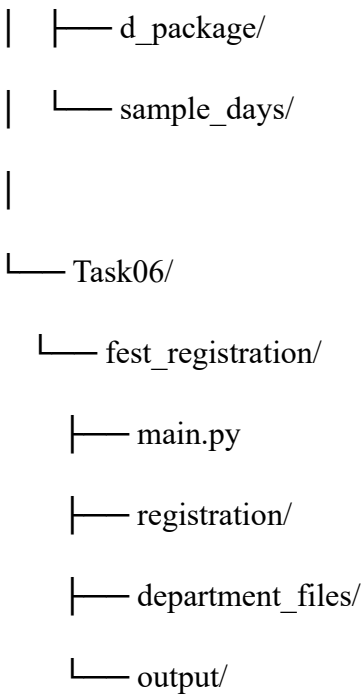
The following programs were executed in the Python virtual environment in GitHub Codespaces.

S.No.	Task	Program executed	Execution result
1	Task 1	task1_mess_bill.py	Mess bill total and rejected records generated
2	Task 2	task2_attendance.py	Attendance percentage, below-75% and damaged-record reports generated
3	Task 3	task3_exceptions.py	Valid records counted and invalid records handled
4	Task 4	task4_file_forensics.py	Return log and error log generated
5	Task 5A	a_single_script.py	Single-script report generated
6	Task 5B	b_module/main.py	Module-based report generated
7	Task 5C	c_three_modules/main.py	Three-module report generated
8	Task 5D	d_package/main.py	Package-based report generated
9	Task 6	fest_registration/main.py	Master list, event files, summary and error log generated

Repository structure:

C04 Assignment 4/

```
│
├── Task01/
│   ├── task1_mess_bill.py
│   └── mess_oct.txt
│
├── Task02/
│   ├── task2_attendance.py
│   └── input/
│
├── Task03/
│   └── task3_exceptions.py
│
├── Task04/
│   ├── task4_file_forensics.py
│   ├── day1.txt
│   ├── day2.txt
│   └── day3.txt
│
├── Task05/
│   ├── a_single_script.py
│   ├── b_module/
│   ├── c_three_modules/
```


d_package/
sample_days/
Task06/
fest_registration/
main.py
registration/
department_files/
output/